

Phase 7 – Project Documentation Phase

1. Project Overview

Project Title: Implement Client Script & UI Policy (Incident)

This project demonstrates the use of ServiceNow UI Policies and Client Scripts to control and validate Incident records. The system dynamically changes field behavior based on Incident Impact and prevents invalid data from being submitted.

2. Problem Statement

Incident records require complete and accurate information for effective incident management. Manual data entry can result in missing or incorrect information.

This project addresses the problem by implementing UI-level controls that automatically manage fields and validate Incident records before submission.

3. Objectives

The main objectives are:

- To improve Incident data accuracy.
- To make Assigned To mandatory for High Impact incidents.
- To automatically set Urgency to High.
- To make Urgency read-only when Impact is High.
- To prevent saving an Incident when required information is missing.
- To prevent State changes through list editing.
- To allow normal updates through the Incident form.

4. Technologies Used

Technology	Purpose
ServiceNow	Development platform
Incident Management	Main application/module
UI Policy	Conditional field control
UI Policy Action	Field behavior configuration
JavaScript	Client-side scripting
Client Scripts	Validation and automation
GitHub	Project repository

5. Modules / Components

5.1 High Impact Control

A UI Policy that is triggered when:

Impact = High

It controls the behavior of Incident fields.

5.2 Assigned To Control

When Impact is High:

Assigned To → Mandatory

5.3 Urgency Control

When Impact is High:

Urgency → Read-only

The onChange script also automatically sets Urgency to High.

5.4 Save Validation

The onSubmit Client Script checks whether Assigned To is filled before saving a High Impact Incident.

5.5 State List Edit Control

The onCellEdit Client Script prevents users from changing State directly from the Incident list.

6. Client Scripts Used

Script	Type	Purpose
Auto set urgency for high impact	onChange	Sets Urgency to High
Prevent save if Assigned To missing	onSubmit	Prevents invalid save
Prevent state change via list edit	onCellEdit	Blocks State list editing

7. Working Process

User creates Incident



Selects Impact



Impact = High



UI Policy activated



Assigned To becomes mandatory



Urgency becomes read-only



onChange sets Urgency = High



User submits Incident



onSubmit checks Assigned To



Valid → Save

Invalid → Error → No Save

8. Testing Summary

The following scenarios were tested:

- High Impact field behavior
- Automatic Urgency update
- Assigned To validation
- Successful Incident submission
- Reverse UI Policy condition
- State list-edit blocking
- Form-based State update

All required test scenarios produced the expected behavior.

9. Benefits

The project provides the following benefits:

- Improves data consistency.
- Reduces manual errors.
- Provides automatic field updates.
- Prevents incomplete Incident submission.
- Improves user experience through dynamic form behavior.
- Restricts unwanted list-based changes.
- Makes Incident data more reliable for management and reporting.

10. Conclusion

The Implement Client Script & UI Policy (Incident) project successfully demonstrates how ServiceNow client-side features can be used to improve Incident data quality.

By combining UI Policies, UI Policy Actions, onChange, onSubmit, and onCellEdit Client Scripts, the system provides dynamic field control, automatic updates, validation, and controlled editing.

The project is lightweight, practical, and demonstrates the application of ServiceNow configuration and JavaScript scripting in Incident Management.